

SLIDING WINDOW

A complete learning chapter for recognizing windows, choosing the right invariant, expanding and shrinking correctly, coding in Java, and solving interview problems systematically.

THE BIG IDEA

A Sliding Window maintains a contiguous range of an array or string. Instead of recomputing every range from scratch, update the window as its left and right boundaries move.

Core payoff: many $O(n^2)$ subarray/substring checks become $O(n)$ when the window state can be updated incrementally.

Chapter map

- Brute force → fixed window → variable window.
- Expand / update state / shrink / record.
- Frequency maps, distinct counts, last-seen indices, and monotonic deque.
- Reusable Java templates and common syntax.
- Problems from foundation to advanced.
- Invariants, edge cases, debugging, complexity, and interview explanation.
- When Sliding Window is the wrong pattern.

MENTAL MODEL

Right explores. Left repairs. The window always represents the current candidate range.

01 · What Is a Sliding Window?

The window is a contiguous interval [left ... right]. As right moves, the window gains information. As left moves, it removes information.

Brute force mindset

```
for (int left = 0; left < n; left++) {  
    for (int right = left; right < n; right++) {  
        // recompute / inspect arr[left..right]  
    }  
}
```

Window mindset

2 1 5 1 3 2
└─────────┘
LEFT RIGHT

Move right → add one element. Move left → remove one element. Do not recalculate the entire range.

Typical signals

- subarray / substring
- longest or shortest contiguous range
- exactly K consecutive elements
- at most K distinct / violations
- maximum or minimum over every fixed-size window

Key condition

The property of the window must be maintainable as elements enter and leave. The pointer movement should have a correctness argument, not just a coding habit.

02 · Pattern Recognition

The strongest signal is a contiguous range: subarray, substring, segment, or consecutive elements.

Problem signal	Think
Exactly K consecutive elements	Fixed-size window
Longest / shortest range satisfying a condition	Variable-size window
At most K distinct / K violations	Window + frequency state
Minimum length satisfying a condition	Expand → shrink → record
Maximum length satisfying a condition	Expand → repair → record
Maximum/minimum in each size-K window	Monotonic deque

Recognition shortcut

- CONTIGUOUS RANGE? → investigate Sliding Window.
- FIXED K? → fixed-size window.
- LONGEST / SHORTEST + CONDITION? → variable window.
- AT MOST K / DISTINCT / FREQUENCY? → window + counts.
- MAX/MIN OF EVERY WINDOW? → monotonic deque.

Important warning

Not every subarray problem is Sliding Window. If the required property cannot be updated efficiently when the left element leaves, consider Prefix Sum, HashMap, Binary Search, DP, or another pattern.

03 · The Universal Framework

Most variable-window problems reduce to four moves: expand → update state → shrink until valid → record answer.

```
int left = 0;

for (int right = 0; right < n; right++) {

    // 1. Add arr[right] to window state

    while (/* window is invalid */) {
        // 2. Remove arr[left] from state
        left++;
    }

    // 3. Window is valid
    // 4. Record answer
}
```

The key decision

What does “valid” mean?

Examples: $\text{sum} < \text{target}$ • $\text{distinct} \leq K$ • $\text{zeros} \leq K$ • required counts satisfied • no repeated character

Invariant

After the shrink loop finishes, the current window satisfies the required condition. The answer is therefore safe to evaluate from that window.

Why the nested while is still $O(n)$

Right moves forward at most n times. Left also moves forward at most n times. They never reset → total pointer movement is $O(n)$.

04 · Pattern A — Fixed-Size Window

The window length is always exactly K. Add the new right element and remove the element that falls out.

```
long sum = 0;

for (int i = 0; i < nums.length; i++) {
    sum += nums[i];

    if (i >= k) {
        sum -= nums[i - k];
    }

    if (i >= k - 1) {
        // window size == k
        // use sum
    }
}
```

Visual

```
      K = 4
    [ 2 1 5 1 ] 3 2
        L R
next → [ 1 5 1 3 ] 2
        L R
add 3 • remove 2
```

Canonical problem — Maximum Average Subarray I

```
double sum = 0;
double best = Double.NEGATIVE_INFINITY;

for (int i = 0; i < nums.length; i++) {
    sum += nums[i];

    if (i >= k) sum -= nums[i - k];

    if (i >= k - 1) {
        best = Math.max(best, sum / k);
    }
}
return best;
```

Complexity: O(n) time, O(1) extra space.

05 · Pattern B — Variable Window: Minimum

For minimum length, shrink aggressively once the window becomes valid.

```
int left = 0;
long sum = 0;
int best = Integer.MAX_VALUE;

for (int right = 0; right < nums.length; right++) {
    sum += nums[right];

    while (sum >= target) {
        best = Math.min(best, right - left + 1);
        sum -= nums[left++];
    }
}

return best == Integer.MAX_VALUE ? 0 : best;
```

Why while, not if?

The first valid window is not necessarily the smallest valid window. Keep moving left while the condition remains true. Every successful shrink is another candidate.

Canonical problem — Minimum Size Subarray Sum

Expand → valid → record → remove left → valid again? record again → stop when invalid.

Critical assumption

For the classic positive-integer sum problem, adding elements increases or preserves the sum. With negative numbers, this monotonic behavior can fail, so this exact template may not be valid.

06 · Pattern C — Variable Window: Maximum

For maximum length, expand while possible. If invalid, shrink only until the invariant is restored.

```
int left = 0;
int best = 0;

for (int right = 0; right < s.length(); right++) {

    // add s.charAt(right)

    while (/* invalid */) {
        // remove s.charAt(left)
        left++;
    }

    best = Math.max(best, right - left + 1);
}
```

Example — Longest Substring Without Repeating Characters

```
int[] freq = new int[128];
int left = 0;
int best = 0;

for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);
    freq[c]++;

    while (freq[c] > 1) {
        freq[s.charAt(left)]--;
        left++;
    }

    best = Math.max(best, right - left + 1);
}

return best;
```

Invariant

After the while loop, every character inside [left..right] has frequency ≤ 1 . Therefore the window contains no duplicate characters.

07 · Pattern D — Frequency / Distinct Windows

Many interview problems define validity through counts: at most K distinct values, enough occurrences, or character-frequency requirements.

At most K distinct — template

```
Map<Integer, Integer> freq = new HashMap<>();
int left = 0;
int best = 0;

for (int right = 0; right < nums.length; right++) {
    freq.put(nums[right],
        freq.getDefault(nums[right], 0) + 1);

    while (freq.size() > k) {
        int x = nums[left++];
        int next = freq.get(x) - 1;

        if (next == 0) freq.remove(x);
        else freq.put(x, next);
    }

    best = Math.max(best, right - left + 1);
}
```

Why remove zero counts?

freq.size() means distinct keys. A value with count zero is no longer part of the window and must be removed.

Useful variants

- At most K distinct → maintain $\text{freq.size()} \leq K$.
- Exactly K distinct → often use $\text{atMost}(K) - \text{atMost}(K-1)$ for counting.
- Known alphabet → `int[26]` / `int[128]` is simple and fast.
- Need the actual range → retain left/right when updating the answer.

08 · Exactly K — A Powerful Counting Trick

$$\text{EXACTLY}(K) = \text{AT MOST}(K) - \text{AT MOST}(K - 1)$$

Why it works

Every subarray with exactly K distinct values is counted by “at most K” but not by “at most K−1”. Subtraction leaves exactly the desired set.

At-most counting template

```
static long atMostKDistinct(int[] nums, int k) {
    if (k < 0) return 0;

    Map<Integer, Integer> freq = new HashMap<>();
    int left = 0;
    long count = 0;

    for (int right = 0; right < nums.length; right++) {
        freq.put(nums[right],
            freq.getOrDefault(nums[right], 0) + 1);

        while (freq.size() > k) {
            int x = nums[left++];
            int next = freq.get(x) - 1;

            if (next == 0) freq.remove(x);
            else freq.put(x, next);
        }

        // Every subarray [start..right] with
        // left <= start <= right is valid.
        count += right - left + 1;
    }
    return count;
}
```

Then

```
long answer =
    atMostKDistinct(nums, k)
    - atMostKDistinct(nums, k - 1);
```

09 · Pattern E — Max / Min in Every Window

When the task asks for the maximum or minimum element inside every fixed-size window, a monotonic deque gives $O(n)$.

Maximum of every window of size K

```
Deque<Integer> dq = new ArrayDeque<>();
int[] ans = new int[nums.length - k + 1];
int out = 0;

for (int i = 0; i < nums.length; i++) {

    while (!dq.isEmpty() && dq.peekFirst() <= i - k) {
        dq.pollFirst();
    }

    while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) {
        dq.pollLast();
    }

    dq.offerLast(i);

    if (i >= k - 1) {
        ans[out++] = nums[dq.peekFirst()];
    }
}

return ans;
```

Deque invariant

Indices are inside the current window and their values are in decreasing order. Therefore the front always stores the maximum.

Complexity: $O(n)$ time, $O(k)$ auxiliary space.

10 · String Windows

Strings are one of the most common Sliding Window categories.

Frequency array

```
int[] freq = new int[26];

for (int right = 0; right < s.length(); right++) {
    freq[s.charAt(right) - 'a']++;

    while (/* invalid */) {
        freq[s.charAt(left) - 'a']--;
        left++;
    }
}
```

ASCII frequency

```
int[] freq = new int[128];
freq[s.charAt(right)]++;
```

Last-seen index — jump left directly

```
int[] last = new int[128];
Arrays.fill(last, -1);

for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);

    if (last[c] >= left) {
        left = last[c] + 1;
    }

    last[c] = right;
    best = Math.max(best, right - left + 1);
}
```

Frequency → counts determine validity. **Last seen** → duplicate location lets you jump left.

11 · Deep Dive — Fixed Window + Frequency Matching

Permutation-in-string and anagram problems combine a fixed-size window with frequency matching.

Core idea

If the pattern has length m , every candidate substring must also have length m . Maintain exactly m characters and compare frequency state.

```
int m = pattern.length();
int[] need = new int[26];
int[] have = new int[26];

for (char c : pattern.toCharArray()) {
    need[c - 'a']++;
}

for (int right = 0; right < text.length(); right++) {
    have[text.charAt(right) - 'a']++;

    if (right >= m) {
        have[text.charAt(right - m) - 'a']--;
    }

    if (right >= m - 1 && Arrays.equals(need, have)) {
        return true;
    }
}
return false;
```

Optimization note

For a fixed alphabet of 26 letters, comparing the arrays is effectively constant time. For larger alphabets, use a map or maintain a mismatch counter.

12 · Java Toolkit for Sliding Window

Keep window state explicit. Most bugs happen when pointer movement and state updates get out of sync.

HashMap

```
Map<Character,Integer> freq = new HashMap<>();

freq.put(c, freq.getDefault(c, 0) + 1);
int count = freq.get(c);
freq.remove(c);
```

String

```
int n = s.length();
char c = s.charAt(i);
int idx = c - 'a';
```

Arrays

```
int[] freq = new int[26];
int[] ascii = new int[128];
Arrays.fill(last, -1);
```

Deque

```
Deque<Integer> dq = new ArrayDeque<>();

dq.offerLast(i);
dq.peekFirst();
dq.peekLast();
dq.pollFirst();
dq.pollLast();
```

Long sums and counts

```
long sum = 0;
long count = 0;
sum += nums[right];
sum -= nums[left];
```

A counting answer can be $O(n^2)$ in magnitude even when the algorithm is $O(n)$, so use long when necessary.

13 · Problem Roadmap

Build the pattern in increasing difficulty.

▢ LEVEL 1 · FIXED WINDOWS

- Maximum Average Subarray I
- Maximum Number of Vowels in a Substring of Given Length
- Find All Anagrams in a String

▢ LEVEL 2 · VARIABLE WINDOWS

- Minimum Size Subarray Sum
- Longest Substring Without Repeating Characters
- Max Consecutive Ones III

▢ LEVEL 3 · FREQUENCY / DISTINCT

- Fruit Into Baskets
- Longest Repeating Character Replacement
- Subarrays with K Different Integers
- Permutation in String

▢ LEVEL 4 · ADVANCED

- Minimum Window Substring
- Sliding Window Maximum
- Substring with Concatenation of All Words
- Count Number of Nice Subarrays

Problem → pattern map

Problem	Pattern	Main lesson
Maximum Average Subarray I	Fixed	Add right / remove outgoing left
Minimum Size Subarray Sum	Variable min	Shrink while valid
Longest No Repeat	Variable max	Frequency + repair
Fruit Into Baskets	At most 2 distinct	Distinct count
K Different Integers	Exactly K	AtMost(K)-AtMost(K-1)
Minimum Window Substring	Variable min + counts	Need vs have
Sliding Window Maximum	Fixed + deque	Monotonic structure

14 · Deep Dive — Minimum Window Substring

This is one of the most important Sliding Window problems because it combines counts, validity, aggressive shrinking, and careful state maintenance.

State

$\text{need}[c] = \text{required count}$ • $\text{have}[c] = \text{current count}$ • $\text{formed} = \text{number of required character counts currently satisfied}$.

```
Map<Character,Integer> need = new HashMap<>();
for (char c : t.toCharArray()) {
    need.put(c, need.getOrDefault(c, 0) + 1);
}

Map<Character,Integer> have = new HashMap<>();
int required = need.size();
int formed = 0;
int left = 0;
int bestLen = Integer.MAX_VALUE;
int bestL = 0;

for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);
    have.put(c, have.getOrDefault(c, 0) + 1);

    if (need.containsKey(c)
        && have.get(c).intValue() == need.get(c).intValue()) {
        formed++;
    }

    while (formed == required) {
        if (right - left + 1 < bestLen) {
            bestLen = right - left + 1;
            bestL = left;
        }

        char x = s.charAt(left++);
        have.put(x, have.get(x) - 1);

        if (need.containsKey(x) && have.get(x) < need.get(x)) {
            formed--;
        }
    }
}

return bestLen == Integer.MAX_VALUE
    ? ""
    : s.substring(bestL, bestL + bestLen);
```

Why formed?

It avoids repeatedly comparing every required character count. Once all required counts are satisfied, the window is valid; then shrink to find the minimum.

15 · Edge Cases & Debugging

Most Sliding Window bugs are state + pointer synchronization bugs.

Checklist

- Empty input.
- $K = 1$.
- $K > n$.
- $K = n$.
- All values identical.
- No valid window.
- Whole input is the answer.
- Answer occurs at the end.
- Duplicates enter and leave.
- Frequency becomes zero — remove the key.
- Counting problems — use long when needed.
- Negative numbers — verify the monotonic assumption.

Debugging grid

```
left = ?  
right = ?  
window = [left .. right]  
state = ?  
valid = ?  
answer = ?
```

When left moves:
What leaves?
Did I remove its state?
Did validity change correctly?

Common mistakes

if instead of while

Minimum-window problems often need repeated shrinking while valid.

Wrong answer timing

Minimum: record while valid before shrinking. Maximum: record after validity is restored.

Forgetting state removal

Every element leaving through left must be removed from the state.

Off-by-one length

Window length = right - left + 1.

Negative sum values

A simple sum-based window may no longer have monotonic behavior.

Resetting unnecessarily

A window should preserve useful state as it moves.

16 · When NOT to Use Sliding Window

Knowing the limits of a pattern is part of mastering it.

Consider another technique when...

- The range is not contiguous.
- The window property cannot be updated efficiently when elements enter/leave.
- Negative numbers break the monotonic behavior required by a simple sum window.
- You need arbitrary range queries — Prefix Sum / Segment Tree may fit.
- The problem is primarily a pair relationship — Two Pointers may fit better.
- You need a decision over a sorted search space — Binary Search may be more natural.

Sliding Window vs Two Pointers

	Sliding Window	Two Pointers
Shape	Contiguous [L..R] range	Two positions / boundaries
Main idea	Maintain a window property	Eliminate, compare, partition, merge
Typical words	substring, subarray, longest, shortest, at most K	pair, sorted, ends, merge
Example	Minimum Window Substring	Two Sum II

17 · Complexity & Correctness

A nested while loop does not automatically mean $O(n)$. Count total pointer movement.

RIGHT

Moves forward at most n times.

LEFT

Moves forward at most n times.

TOTAL

At most about $2n$ pointer moves $\rightarrow O(n)$.

Typical complexities

Pattern	Time	Extra space
Fixed window	$O(n)$	$O(1)$ or state size
Variable window	$O(n)$	$O(1)$, alphabet, or distinct
Frequency map window	$O(n)$ average	$O(\text{distinct})$
Exactly K via two calls	$O(n)$ average	$O(\text{distinct})$
Monotonic deque	$O(n)$	$O(k)$
Sort + window	$O(n \log n) + \text{scan}$	Depends

Correctness checklist

- Window is always contiguous.
- Every entering element updates state.
- Every leaving element reverses its state update.
- Shrink condition exactly matches invalidity.
- Answer is recorded at the correct moment.
- Pointers move monotonically.

18 · How to Explain It in an Interview

Make the invariant visible.

Five-step script

- Observation: “The problem asks about a contiguous range, so I can maintain that range as a window.”
- State: “I maintain the information needed to decide whether the window is valid.”
- Expand: “Right moves forward to include new elements.”
- Repair: “When invalid, left moves forward and removes elements until the invariant is restored.”
- Complexity: “Both pointers move only forward, so total pointer movement is $O(n)$.”

Example — Longest substring without repeating characters

“I maintain a contiguous substring with no duplicate characters. Right expands the window and increments a frequency count. If a duplicate appears, left moves forward and decrements counts until the window is valid again. Then I update the maximum. Each character enters and leaves at most once, so the time is $O(n)$.”

19 · Sliding Window Decision Tree

Use this before coding.

Question	If YES	Next move
Contiguous subarray / substring?	Potential window	Choose fixed or variable
Fixed K?	Fixed window	Add right / remove outgoing left
Longest / shortest condition?	Variable	Define valid / invalid
Counts / distinct values?	Frequency state	Map or array
Exactly K counting?	AtMost trick	$K - (K-1)$
Max/min each fixed window?	Monotonic deque	Maintain candidate indices
Can state update on entry/exit?	Fits window	Write invariant
Cannot update / monotonicity fails?	Maybe not	Try another pattern

Write these four lines before code

1. Window = [left .. right]
2. State = _____
3. Valid when = _____
4. Shrink while = _____

20 · Final Cheat Sheet

FIXED WINDOW

Add right → if size > K remove left → when size == K record

VARIABLE — MINIMUM

Expand → while valid record → remove left → repeat

VARIABLE — MAXIMUM

Expand → while invalid remove left → record maximum

FREQUENCY WINDOW

Add count → check condition → remove count when left exits

EXACTLY K

exactly K = atMost(K) – atMost(K–1)

MAX/MIN EVERY WINDOW

Fixed window + monotonic deque

STRING WINDOW

freq[26/128], HashMap, or last-seen indices

THE SIX QUESTIONS

- Is the target range contiguous?
- Fixed K or variable?
- What makes the window valid?
- What state must be added/removed?
- When exactly should I shrink?
- When exactly should I record the answer?

Practice progression

Fixed window → Minimum Size Subarray Sum → Longest No Repeat → Frequency/Distinct → Exactly K → Minimum Window Substring → Sliding Window Maximum

FINAL RULE

Do not memorize “left++”. Memorize what becomes invalid, what leaves the window, and how the invariant is restored.

End of chapter • Sliding Window